ConfigStream

Scalable Configuration Change Storage and Streaming Analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

First milestone checkpoint · 26 October 2026

The problem: configuration changes over time

Router configuration: version 10

```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

What changed, when, and on which asset?

A current configuration file alone cannot answer all three.

Illustrative input from the executable demo: processing/demo.py.

What ConfigStream provides

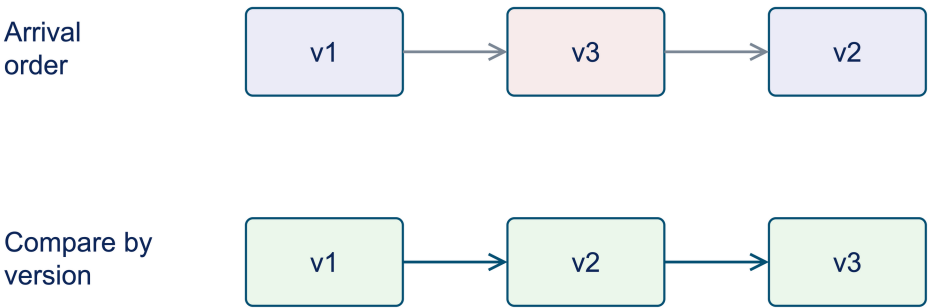


A searchable history of configuration changes

One asset ID links its versions, differences and alerts.

Implemented contracts: schemas/; processing/; api/routes/documents.py.

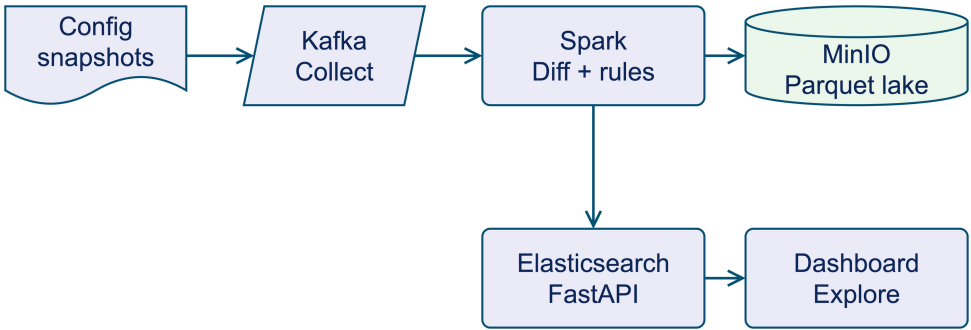
Late arrivals should not create the wrong diff



Compare **v1** → **v2** and **v2** → **v3**, once both endpoints are available.

processing/state.py; spark/streaming/stateful.py. Online repair is bounded by retained state.

System architecture



Parquet keeps the durable history. Search and the dashboard provide read access.
Historical Spark analytics reads the committed Parquet datasets.

Configuration path: spark/streaming/outputs.py; analytics/main.py; api/; frontend/.

Data is ready for development and replay

Controlled examples

A deterministic generator creates snapshots, retries and late arrivals. Useful for checking whether the result is correct.

TerraDS configuration history

6,505	file histories
10,188	accepted versions
3,683	adjacent version pairs

Public Git history of infrastructure code.

Large datasets stay outside Git; source manifests and result receipts are versioned.

generator/; workloads/results/terrads/history-summary.json. Git changes are not deployment events.

Implemented: differences and rule alerts

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Rules detect

Logging disabled
High severity

Telnet enabled
Critical severity

Repeated delivery must not invent another logical change.

processing/demo.py; processing/diff.py; rules/default.yaml; tests/unit/test_rules.py.

Executable code is available in Git

Repository

`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

Small demo, no services required

```
make setup
make processing-demo
```

178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

github.com/kongwoang/ConfigStream; Makefile; tests/unit/test_state_logic.py, test_diff.py, test_rules.py.

A complete local replay has been recorded

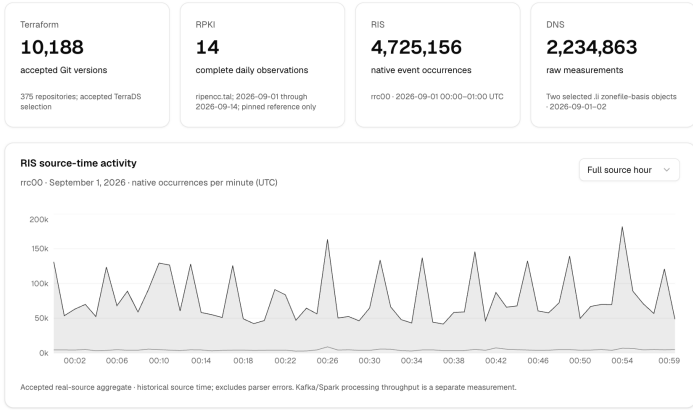


29.4 MB of raw and processed Parquet committed.

This demonstrates the integrated path on the selected local workload.

Recorded acceptance: docs/phase12-acceptance.md. 29,402,997 bytes; not a new replay or a scale benchmark.

Dashboard preview



Actual frontend captured on 25 October 2026. This overview displays the accepted evidence bundle, not a live replay.

Checkpoint readiness

Requirement	What the group can show
Git + executable code	Repository, runnable demo and automated tests
Data ready	Controlled inputs, TerraDS history and source manifests
Architecture ready	Kafka, Spark, Parquet, search API and dashboard

The core workflow is implemented and demonstrated locally

Next milestone

1. **Rehearse the integrated demo**
   Start the services, send a change, inspect the stored result.
2. **Evaluate larger workloads**
   Measure throughput, latency and storage with clear units.
3. **Strengthen recovery evidence**
   Repeat failure and restart checks for the latest revision.

Current evidence is local / single-node; distributed scale and HA remain open.

Thank you — questions?